Reason. Plan. Act. Physical AI Workshop

Lab 1.1: Task Planning

Guy Azran and Sarah Keren

The Taub Faculty of Computer Science
Technion - Israel Institute of Technology

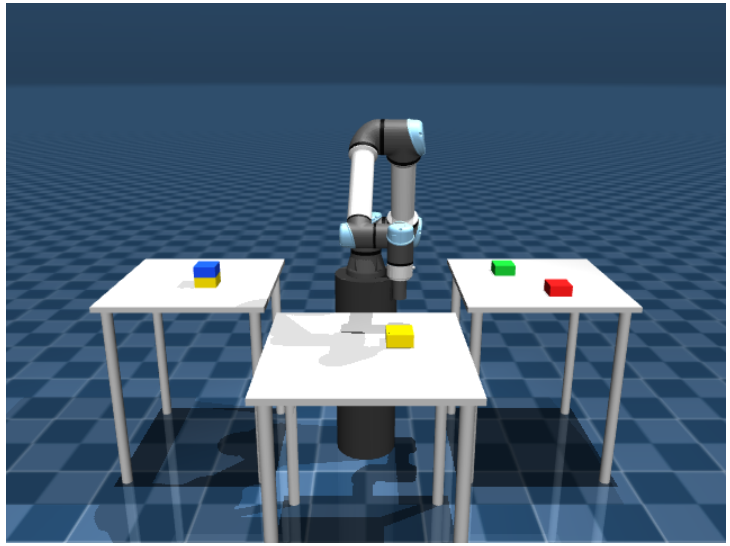


Reason, plan, act

Guy Azran and
Sarah Keren

Robots that carry out multi-step tasks need two kinds of reasoning:

- **Task planning**: which actions, in which order?
  (symbols: blocks, tables, `pick-up`)
- **Motion planning and control**: how to move to carry them out?
  (numbers: joint angles, torques)



This lab is about the first.

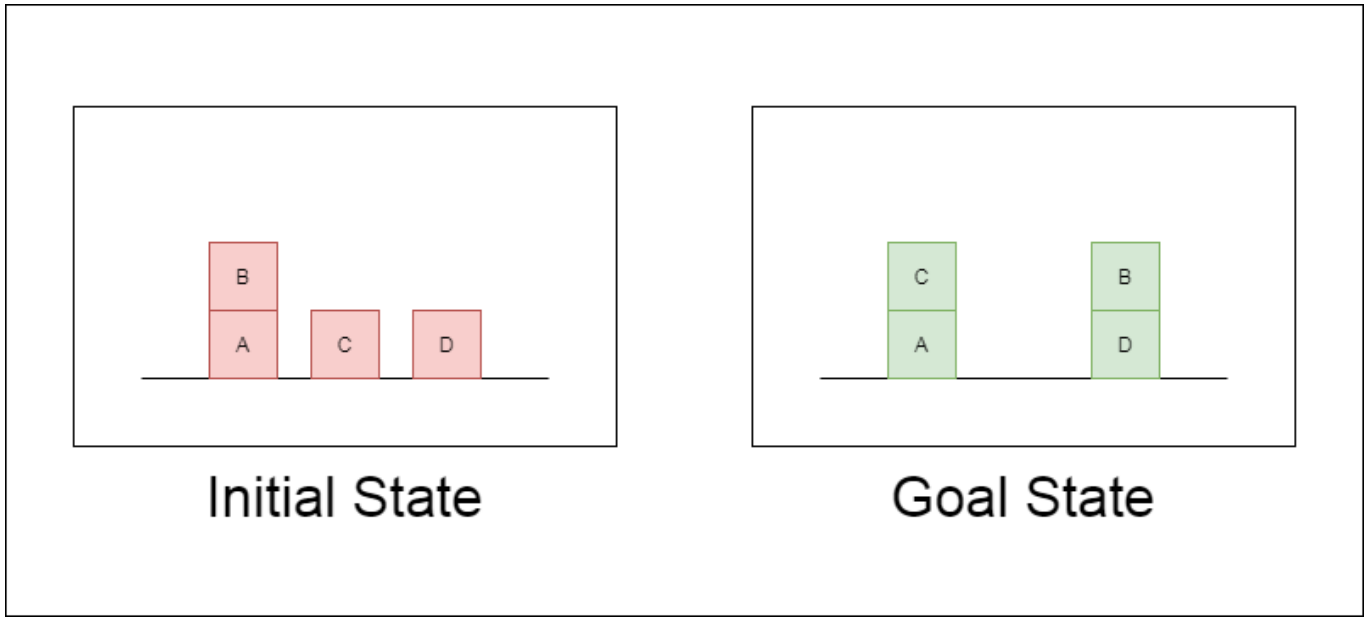
1.1 Task planning	2.1 Motion planning	3.1 State estimation	4.1 Integrated TAMP
1.2 Inverse kinematics	2.2 Control	3.2 Decoupled TAMP	4.2 VLAs

Today

Learn	The blocks world in PDDL, solved with a planner
Do	Task 1: walk through the state space
Learn	How planners search: BFS, DFS, A*, heuristics
Do	Task 2: design a heuristic
Do	Task 2.1: compare heuristics on harder problems
Learn	Fast Downward, optimality, executing plans in PDDL-Gym
Do	Task 3: an optimal plan versus a quick one
Learn	The N-Table Blocks World
Do	Task 4: extend the model
Learn	From plans to robots: why we need TAMP

Running example: the blocks world

Guy Azran and
Sarah Keren



- Actions: `pick-up`, `put-down`, `stack`, `unstack`
- Each action has **preconditions** and **effects**: e.g., a block can be picked up only if it is clear and the hand is empty

Solve it by hand. How many actions did you need? Is there a shorter plan?

PDDL: the domain

The **domain** holds what is shared by all problems: the **types** of objects (`block`, `robot`), the **predicates** (`on`, `ontable`, `clear`, `handempty`, `holding`) and the **actions**:

```
(:action pick-up
    :parameters (?x - block ?r - robot)
    :precondition (and (clear ?x) (ontable ?x) (handempty ?r))
    :effect (and (not (ontable ?x))
                 (not (clear ?x))
                 (not (handempty ?r))
                 (holding ?x))
)
```

- Parameters are variables, filled in with objects of the problem
- Effects add facts, or delete them with `not`
- Anything the effect does not mention stays as it was

PDDL: the problem

Guy Azran and Sarah Keren

The **problem** is one instance of the domain:

- objects
- initial state
- goal

The goal is *partial*: every state in which `(on c a)` and `(on b d)` hold is a goal state.

```
PROBLEM = """
(define (problem blocks-4)
    (:domain blocks)
    (:objects
        a b c d - block
        rob - robot
    )
    (:init
        (ontable a) (on b a) (clear b)
        (ontable c) (clear c)
        (ontable d) (clear d)
        (handempty rob)
    )
    (:goal (and (on c a) (on b d)))
)
"""
```

Why is the robot an object? Would the model be wrong without it?

Solving it with an off-the-shelf planner


```python
from unified_planning.shortcuts import OneshotPlanner

with OneshotPlanner(name="fast-downward") as planner:
    result = planner.solve(problem)

print("status:", result.status.name)
print("plan:")
for i, action in enumerate(result.plan.actions):
    print(f"  {i + 1}. {action}")
```

```
status: SOLVED_SATISFICING
plan:
  1. unstack(b, a, rob)
  2. stack(b, d, rob)
  3. pick-up(c, rob)
  4. stack(c, a, rob)
```

UPF: one Python interface to many planners, here **Fast Downward**.
Validate before you trust: `PlanValidator`, `SequentialSimulator`.

Task 1: walk through the state space

Guy Azran and
Sarah Keren

⚒ **Task 1**: Use the simulator to list all the actions that are applicable in the initial state. Then take one of them, apply it, and list the applicable actions in the resulting state.

Hint: `simulator.get_applicable_actions(state)` yields `(action, parameters)` pairs. To turn such a pair into an action instance that `simulator.apply` accepts, use `simulator.apply(state, action, parameters)`.

```
with SequentialSimulator(problem) as simulator:
    state = simulator.get_initial_state()
    # your code here
```

- How many actions are applicable in the initial state?
- How many states does this problem have? What about 10 blocks?

How does a planner find a plan?

Guy Azran and
Sarah Keren

States and actions form a huge directed graph. A plan is a path from the initial state to *some* goal state.

Best-first search expands the node with the lowest priority $f(n)$:

search	$f(n)$
BFS	$depth(n)$
DFS	$-depth(n)$
A*	$g(n) + h(n)$
greedy	$h(n)$

on our problem	plan	expanded
BFS	4	41
DFS	22	40
A*, goal heuristic	4	22

$g(n)$: the cost so far.

$h(n)$: an estimate of the cost to the goal, the **heuristic**.

Heuristics: a sense of smell

A heuristic $h(n)$ estimates $h^*(n)$, the cost of the cheapest path from n to a goal.

goal-aware	$h(n) = 0$ in goal states	
safe	$h(n) = \infty$ only if $h^*(n) = \infty$	
admissible	$h(n) \leq h^*(n)$	$\Rightarrow$ A* is optimal
consistent	$h(n) \leq c(n, n') + h(n')$	$\Rightarrow$ no re-expansions

The trade-off at the heart of planning research: an **informative** heuristic expands fewer nodes, a **cheap** one costs less per node.

Task 2: design a heuristic

Guy Azran and
Sarah Keren

⚒ **Task 2**: Implement a heuristic for our problem. Try to minimize the number of expanded nodes.

```
def your_heuristic(state, problem):
    # your code goes here
    return 0
```

Some directions:

- count the goal facts that do not hold yet
- which blocks are “in the way”?
- the **delete relaxation**: ignore delete effects and solve the much easier relaxed problem

Is your heuristic goal-aware? Admissible? Consistent?

Task 2.1: compare heuristics on harder problems

Guy Azran and
Sarah Keren

⚒ **Task 2.1**: compare heuristics on a harder problem.

To better understand the effectiveness of different heuristics, let's compare them using a more complex problem. Below is a problem instance with 6 blocks that requires multiple stacking and unstacking actions. Run BFS, A* with the goal-count heuristic, A* with the relaxed-layers heuristic and greedy best-first search with the relaxed-layers heuristic on it, and compare plan lengths, expanded nodes and running times.

Then, write an even harder problem of your own (add a block or two, or make the goal a full tower) and see which searches can still solve it in reasonable time.

```
hard_problem = PDDLReader().parse_problem_string(DOMAIN,
HARD_PROBLEM)

### Your code here, use as many cells as you want.
```

Is the heuristic that expands the fewest nodes also the fastest?

Heuristics in a state-of-the-art planner

Guy Azran and
Sarah Keren

Seven blocks into one tower, with four Fast Downward configurations:

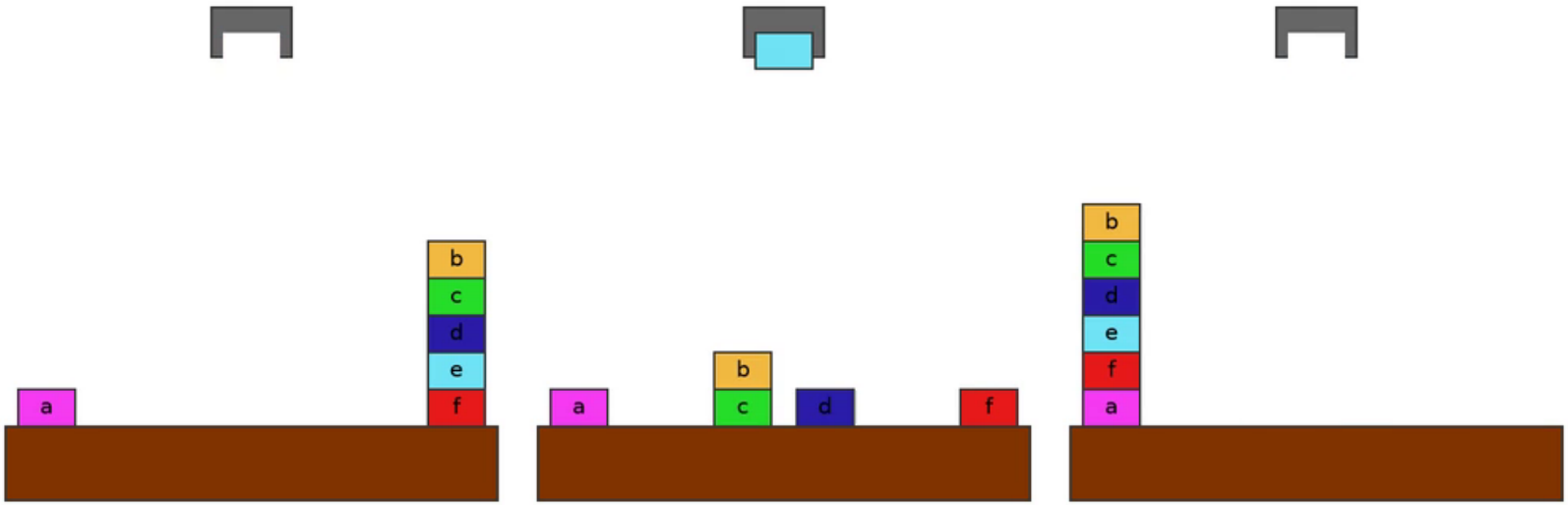
configuration	plan	expanded
A*, blind heuristic ($h = 0$)	20	55 609
A*, LM-cut (admissible)	20	245
greedy, FF (inadmissible)	24	42
lazy greedy, FF (the default)	34	71

- **Satisficing** planners look for *a* plan, quickly
- Ask UPF for an **optimality** guarantee, and it chooses A* with LM-cut

Why would anyone use an inadmissible heuristic?

Executing plans in PDDLGym

Guy Azran and
Sarah Keren

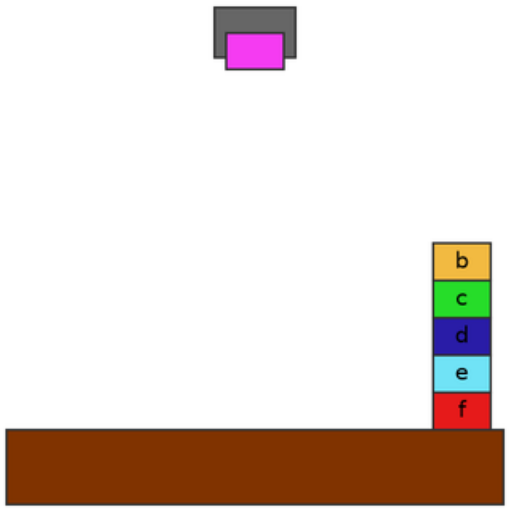


- PDDLGym turns PDDL problems into Gym environments: `reset()`, `step(action)`
- We translate between UPF and PDDLGym actions, as we will later translate between a planner and a robot
- Fast Downward's default plan for this problem has 24 actions. Any redundant moves?

Task 3: an optimal plan versus a quick one

⚒ **Task 3**: Solve the same problem with an optimality guarantee (see the previous section), execute the plan, and compare the two videos. Is the difference what you expected?

```
env.reset()
# your code here
```



- Use the optimality guarantee of the previous section
- Compare the plan lengths, and the two videos

The N-Table Blocks World

Guy Azran and
Sarah Keren



```
(define (domain n-table-blocks-world)
  (:requirements :strips :typing)
  (:types block color table)
  (:predicates (on ?b1 ?b2 - block)
               (ontable ?b - block ?t - table)
               (clear ?b - block)
               (handempty)
               (holding ?b - block)
               (bcolor ?b - block ?c - color))
```

Blocks on n tables, each with a color. The NTBW stays with us for the rest of the workshop.

What does this model ignore? Picking up a yellow block from any table makes `(holding ?b)` true, but the costs and risks of doing so are very different.

Task 4: extend the model

Guy Azran and
Sarah Keren

⚒ Task 4:

1. Try to capture some of the complexities that would occur in a real-world NTBW that are not captured in the PDDL model we gave above.
2. As another challenge, can you think of a representation that will allow specifying that there should be a block with a specified color on a table, instead of referring to labeled blocks (e.g., "a yellow block on table 1")?

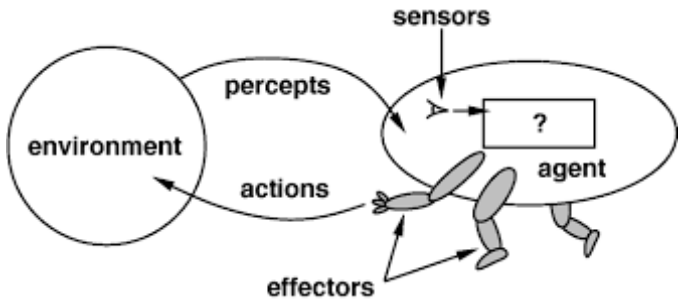
```
NTBW_DOMAIN_EXTENDED = NTBW_DOMAIN  # your code here: replace with
your own domain string
NTBW_PROBLEM_EXTENDED = NTBW_PROBLEM  # your code here: replace
with your own problem string
```

- Useful requirements: `:negative-preconditions`, `:equality`, `:conditional-effects`, `:existential-preconditions`, `:action-costs`
- Experiment in the planning.domains editor

From plans to robots

What does it take to execute `pick-up(b2, t3)`?

- find table `t3` and block `b2`
- choose a configuration to grasp from
- get there without hitting anything
- check that it worked, and adapt if not



perception, control, actuation

Task planning alone misses physical feasibility. Motion planning alone misses the goal. Combining them is **task and motion planning (TAMP)**.

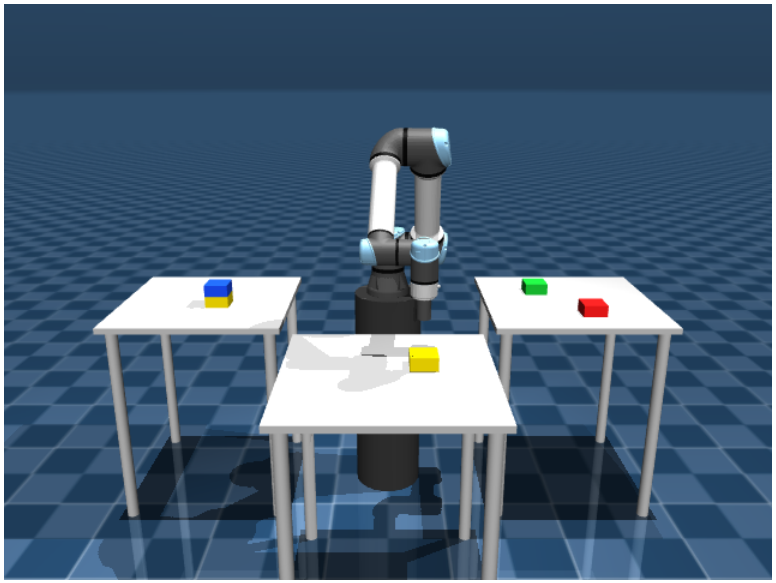
Get to work!

Summary

In this tutorial we

- modeled the blocks world in PDDL: a domain and a problem
- solved it with the Unified Planning Framework and Fast Downward, and validated the plan
- implemented best-first search: BFS, DFS, A* and greedy search
- designed heuristics, trading informativeness against cost
- executed quick and optimal plans in PDDLGym
- extended the model to the N-Table Blocks World
- saw why a robot needs more than a task plan: TAMP

What comes next



The NTBW in the MuJoCo simulator

- **Kinematics**: from joint angles to gripper poses, and back
- **Motion planning**: moving without collisions
- **Control**: following the motions
- **State estimation**: from images to facts
- **TAMP**: putting it all together